PROJECT BRIEFING

Maltese Snow War

Hold, Dodge, and Throw!

Code structure, runtime flow, P2P smoothness, and the big-snowball pickup.

A browser remake of the 1998 Flash game SnowCraft, with moonlab's Line Puppy dogs: Maltese vs golden retrievers. Vs AI (Easy / Hard) and 1v1 WebRTC rooms.

Internal architecture document · August 2026
Source of truth: `github.com/rayony/maltese-snowwar` · live: maltese-snowwar.grok.me

1. What this is

Maltese Snow War is a canvas snowball fight in the browser. Hold a dog to move, release to throw, pack snow between shots. Vs AI still uses tap-near / hold-far. Versus uses auto-aim plus a fixed range and speed. White Maltese (red scarves) stand on the right; brown retrievers (party hats) on the left. The host always plays the Maltese. The guest is mirrored and plays the retrievers.

Two-hit bury, ellipse forts, pack-snow cooldown, a 3-2-1 countdown, and a mid-fight **big snowball** pickup. Unofficial fan tribute to Nicholson NY's SnowCraft (1998) and moonlab's Line Puppy illustrations.

There is no dedicated game-simulation server — the host's browser is the authority; a small signaling server only helps the two browsers find each other.

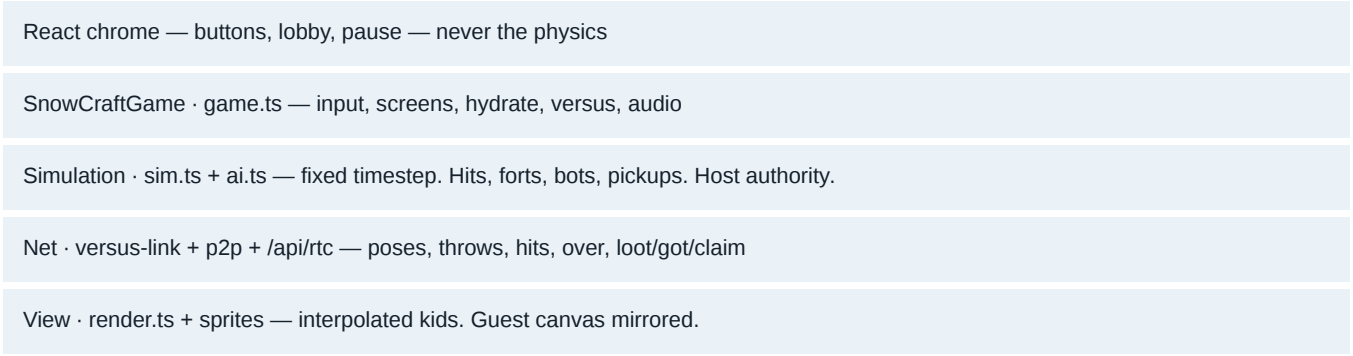
2. Stack

Layer	Choice	Why
UI shell	React 19 + TanStack Router + Tailwind	Title, lobby, pause, HUD. Canvas sits underneath.
Game	TypeScript, Canvas 2D, fixed 1/60 s	Deterministic-enough sim; render is a view, not the clock.
Net	WebRTC data channels + HTTP/SSE fallback	Peer-to-peer after join. TURN only if NAT blocks UDP.
Host	Vite + Nitro (Vercel preset)	SSR title, `/api/rtc` signaling.

3. Repository map

Almost all game logic lives under `src/game/`. React is thin chrome around one `SnowCraftGame` instance.

Path	Role
`src/components/game/SnowCraft.tsx`	Title, Easy/Hard, lobby, pause, HUD, QR, language, pickup banners.
`src/game/game.ts`	Orchestrator: screens, input, audio, net, loop, loot/claim.
`src/game/sim.ts`	Pure world step: throws, collisions, forts, bury, pickups.
`src/game/ai.ts`	Bot brains. Easy vs Hard; Defend / Attack allies.
`src/game/render.ts`	Canvas paint. Guest view is mirrored. Gold-rim big orb.
`src/game/net.ts`	Wire types, pack/apply pose & snapshot, binary pose codec.
`src/game/versus-link.ts`	P2P + HTTP bus, seq, ping, clock offset.
`src/game/assets.ts`	Boot (title) then rest (poses) hydrate with progress.
`src/game/i18n.ts + locales/`	EN + 繁中 first; 简 / ja / ko on select.



Data flows down: pointer → game.ts → sim (host) or local prediction (guest). Paint always reads `GameState`. The guest never decides a bury or a winner.

4. Screen flow

Title (boot spinner + English title assets) → mode select → pack action sprites (progress) → 3-2-1 → fight → over / rematch.

Title is React-only after boot: Play vs AI / Friend are real buttons. Action sprites load only after Easy/Hard, or after both PvP seats are in the room (packed handshake). Nothing streams in mid-match. Countdown lets dogs walk but not pack snow. Vs AI can return to title from pause. Vs Friend stays in the room on defeat so rematch does not need a new code.

4.1 Versus join

Host mints a 6-letter code. Guest types it or scans the QR. Neither side starts until both have finished packing sprites. Host then sends `start`. Guest renders the yard flipped so the retrievers sit on the guest's right.

5. Making P2P feel smooth

Host-authoritative P2P is uneven: the host is 0 ms from the sim, the guest is one RTT away. For a friend-code 1v1 we delay the host a little and predict on the guest. No GGPO rollback and no dedicated sim server.

5.1 What goes over the wire

Message	Channel	When	Payload
`pose`	Unreliable + reliable copy (binary)	~14–20 Hz	id, x, y, vx, vy, facing, state, hp + live balls
`allypose`	Unreliable + reliable copy	Guest ~15–25 Hz	Guest-yard kids x,y
`throw / allythrow`	Reliable + HTTP copy	On release	id, origin, velocity, team, t0, big?
`hit`	Reliable + HTTP	On damage	id, hp, heavy. Host only.
`loot / got / claim`	Reliable + HTTP	Orb spawn / pickup / tap	xy life / team shots ttl / claim
`snap`	Unreliable + reliable copy	Every 1.2 s + start	Full field for recovery
`over / start / rematch / packed`	Reliable + HTTP	Events	Winner; over repeats 0.35 s
`input`	Move dual-sent; down/up reliable	Guest stick + throw	x, y, hold, vx, vy, at
`ping / pong`	Unreliable + reliable copy	2 s	t0, t1 → RTT + clock offset

Seat	Feel
Host (Maltese)	Move instant. Throw shows a ghost ball at once; real collision waits `clamp(RTT/2, 30–80 ms)` from an EMA RTT.
Guest (retriever)	Own move/throw are local. Opponent pose interpolates 55–70 ms. HP/SFX wait for host `hit`.

5.2 Catalogue (live)

1. **Host input delay** — throws queued by clamp(RTT/2, 30–80) ms. Movement stays immediate.
2. **Host ghost ball** — paints immediately, collides after the delay.
3. **Guest local move & throw** — no round-trip. Pose does not overwrite the grabbed dog.
4. **RTT-tiered hit juice** — cosmetic only. HP/bury/winner are host-only.
5. **Remote interpolation** — guest paints Maltese 55–70 ms in the past.
6. **Clock offset** — ping t0 / pong t0,t1. Guest inputs carry `at`.
7. **Keyframes do not clobber balls** — 1.2 s snap repairs kids/HP, keeps local balls.
8. **Two channels + pose copy** — unordered state + ordered event. Poses also copy onto reliable (LAN/iOS drop).
9. **Tiny over packet** — `{ winner }` resent 0.35 s.
10. **Packed handshake** — match waits until both have pose sprites.
11. **ICE / TURN / relay HUD** — STUN first; ICE restart; SSE poll fallback. HUD: direct vs relay, RTT, fps.

12. **Guest-local presentation** — fidget, walk, aim, particles, SFX, 3-2-1 are not networked.
13. **Ally bots on the guest** — Defend/Attack step locally; `allythrow` + `allypose` go up.
14. **Live EMA RTT** — 0.8/0.2 drives host delay, interp window, hit-FX tier.
15. **Throw timestamps (t0)** — host catch-up-simulates a late guest throw (~200 ms).
16. **Versus throw profile** — auto-aim; range 520 / speed 440; tap = hold. Pack 0.92 s.
17. **Binary pose** — ArrayBuffer pose (magic SC) to cut JSON GC. Events stay JSON.
18. **Guest disconnect takeover** — if the DataChannel stays down ~3 s mid-match, host local AI (`ai.ts`) takes the guest team. No lobby bounce.

5.3 Versus combat recipe

Knob	Vs AI	Versus
Aim	Nearest (Hard may scatter / shoot back)	Nearest foe, both seats
Charge	Tap near, 1.2 s hold = full field (×1.2 if big)	None — tap = hold
Range / speed	58–820 px / 360–500 (Hard ×2)	520 / 440
Pack	0.92 s (star 0.32 s)	0.92 s
Easy retrievers	Own half, no backward / left shots	—
Hard retrievers	May cross midfield and shoot backward	—

6. Big snowball pickup

A team-level buff, not bound to the dog who touched the orb. Player-initiated throws consume it; AI throws do not.

- **Spawn** — mid-fight, on a cooldown (~8–12 s after the last orb). Host is authority. Guest sees `loot`.
- **Look** — white snowball, gold glowing rim only (not a gold fill, not a stretched face sprite). Field orb is the same radius as a thrown big ball (~90% of the earlier oversized size).
- **Collect** — walk any living dog onto it, *or tap/click the orb* (PvP guest sends `claim`; host grants green). Second pickup refills 3 shots / 10 s.
- **Use** — next 3 *user* throws, or 10 s, whichever first. 2 HP. Fly at 0.8×. Vs AI: 1.2× hold to reach max range. PvP has no charge, so only the slower flight applies.
- **Clash** — hit by a normal ball → shrinks to a normal ball (then HP−1). Two bigs shatter both.
- **HUD** — bottom-center. Field: “A big snowball appeared! Grab it!” Held: shots + time bar. PvP foe held: blinking red “Opponent has the big snowball!”
- **Cheat** — double-tap the X vs Y HUD (host/solo) to spawn one orb. Both AI and the player can collect it.

> Net: host broadcasts `loot` (spawn/despawn) and `got` (team, shots, ttl). Guest tap is `claim`. Throws carry `big?` so the guest paints the gold rim without waiting for a snap.

7. Loading strategy

SSR paints a spinner until React hydrates. Then `loadBootAssets`: English title background, Caveat, idle dog heads — with a progress bar. Play vs AI / Friend stay clickable. After a mode is chosen, `hydrateRest` loads pose sheets (progress on the packing screen). Language packs other than EN (and 繁中 already in the bundle) load only when that language is selected. BGM starts on the first armed tap, not on boot.

8. Vs AI (for contrast)

Easy: original pacing, nearest-target bots, forts are cover only, Maltese revive next heat; retrievers stay on their half and only throw forward. Hard: bot move ×3, smarter dodge, forts 10 HP, no Maltese revive, Maltese move ×2 and both teams’ snowballs ×2; retrievers may cross midfield and shoot backward. Same sim as PvP; no net. Difficulty is chosen on a gate after Play vs AI, which is also when pose sprites load.

9. Trust and secrets

TURN username/credential live in server env, never in git. P2P gameplay is host-honest: a cheating host can still lie. Fine for friends; not for ranked. Cheats (star mode, instant orb) are local/host only.

10. Credits

Game: Nicholson NY, SnowCraft, 1998. Characters: moonlab / Line Puppy (fan illustration). Fight feel referenced jeffreywilbur/snowcraftjs. Produced by Gary.TC. Libraries listed in-app at /credits.

End of briefing. Source of truth is the GitHub repo (beta / main). This PDF describes the live stack as of 27 August 2026: binary pose, EMA RTT, loot/claim, gold-rim big ball, staged boot loader, guest bot takeover.